

XRPL Validator — Daily Engineering Report • 2026-08-19

Compiled retrospectively on 2026-09-23 from the git history. Scope: XRPL work only — the git history of [xrpl-validator](#) and the XRPL Rust SDK, the XRP Ledger itself (amendments) and XRPL research.

Summary

- **18 commits** (xrpl-validator 18)

Engineering changes

- **fix(xrpl-ledger): TrustSet destination rules — no new lines at pseudo-accounts or opted-out issuers** (442c34c , xrpl-validator) TrustSet.cpp:220-278, in rippled's order, both in preclaim before any reserve arithmetic: 1. lsfDisallowIncomingTrustline on the LimitAmount's issuer refuses a NEW line with tecNO_PERMISSION; fixDisallowIncomingV1 still allows adjusting an EXISTING one. 2. Pseudo-account destinations: an AMM account accepts a new line only for its own LP token on a non-empty pool (wrong currency tecNO_PERMISSION, empty pool ...
- **fix(xrpl-ledger): AMMWithdraw judges the DERIVED burn against the position, and the one-asset minimum** (8c5e86b , xrpl-validator) Two rules from the same file, cleared three specimens: 1. rippled's withdraw() core (AMMWithdraw.cpp:515-520) judges the ACTUAL burn against the LP's holding for EVERY mode — "failed to withdraw, invalid LP tokens". We judged only an explicit LPTokenIn; the amount-driven modes (tfTwoAsset, tfSingleAsset) derive their burn and sailed past. #106363879 55335BB9: 538.5 XRP derives 396924.53 LP against 369936.01 held — ...
- **fix(xrpl-ledger): AMMCreate preclaim — duplicate pool, reserve, funds, LP-token sides** (24d983a , xrpl-validator) AMMCreate::preclaim in rippled's order (AMMCreate.cpp:100-186); we checked nothing but account existence: 1. A pool for the pair already exists — tecDUPLICATE. #106118993 D98E76D5 recreates XRP/USD; we answered tecNO_PERMISSION from do_apply's own collision handling. The probe now asks ledger_entry amm for the pair (a fee-only meta never names the standing pool). 2. xrpLiquid(account, +1) <= 0 — the LPToken line's ...
- **fix(xrpl-ledger): AccountDelete has no OwnerCount rule — the directory walk decides** (976d6e3 , xrpl-validator) rippled's preclaim never looks at OwnerCount: the owner-directory walk answers techAS_OBLIGATIONS on any entry without a nonObligationDeleter, whatever the count says. Our count-first gate returned tecNO_PERMISSION before the walk ever ran — right refusal, wrong code, from a rule rippled never had. #106066467 248BF6E3.
- **feat(xrpl-ledger): DirectStepI stage 1 — pure-account strands ripple for real** (f13ad50 , xrpl-validator) docs/DIRECTSTEP-DESIGN.md stage 1: a Payment path made only of account elements becomes a strand of DirectStepI hops executed by a dedicated rev/fwd walker — the engine's first value delivered across mutual trust lines instead of books.
- **feat(xrpl-ledger): DirectStepI stage 2 — direct runs composed with book hops** (87bffeb , xrpl-validator) The mixed five flow: a Payment path mixing account elements with currency/issuer elements becomes segments

[Run|Book] (mixed_layout — the same toStrand normalization, shared verbatim with the probe's hydration), sized right-to-left (books via size_book_hop, factored intact out of reverse_requirements; runs via run_rev) and flowed forward (books via cross_engine_to with the value riding IN FLIGHT through the sender; ...

- **feat(xrpl-ledger): the default path of a cross-issuer same-currency payment is a direct strand** (4771bdd , xrpl-validator) toStrand builds the default strand src → [issuers] → dst for the shape terminal_is_ripple_step used to refuse wholesale, and lets the strand's own checks decide. Now we do too, gated to EXACTLY the classic-refused shape so nothing is modeled twice: #106065267 98C5B11E (1700 ZAR, src-issue → dst-issue, NO Paths) delivers over the one mutual line; #106336831's terNO_LINE refusal is preserved by the line check. The ...
- **fix(xrpl-ledger): offer crossing judges the pass with the OUT issuer's transfer rate composed** (0d4ce0f , xrpl-validator) BookOfferCrossingStep builds its CLOB quality function with WaiveTransferFee::No, so "Path rejected by limitQuality" fires on filed x trOut while qualityUpperBound (admission) deliberately waives the fee. Same rule in the walk's encoding: crossing_judge_threshold tightens the taker's threshold by rate(TakerPays issuer) for CLOB fills — AMM fills waive (OfferType::Amm), a bridge leg-B offer owned by the taker waives ...
- **fix(xrpl-ledger): single-path crossing lets the AMM emit unbounded when the limit beats the LOB tip** (8d8079d , xrpl-validator) Two coupled rules from rippled's fixAMMv1_1 crossing path, pinned by FFI receipts (TRYAMM/ADMIT/STEPREV/STEPFWD probes on the vendored 3.2.1): 1. BookOfferCrossingStep::qualityThreshold (BookStep.cpp:475-481): in SINGLE-path crossing, when limitQuality > the leg's LOB tip quality — a unit-blind Quality compare, so on bridge legs it contrasts cross-pair rates and drops-inflation decides — getAMMOffer(nullopt) emits ...
- **fix(xrpl-ledger): a skipped self-offer stays the tail AMM anchor; the trOut judge compose was wrong** (6f32591 , xrpl-validator) Two crossing rules, both pinned by FFI receipts and gated together (ammunb3: CORPUS 1461/1461 FRESH 3114/3114 NEW 5437/5437 BATCH2 2524/2524, census 0): 1. A self-offer SKIPPED without consumption stays the tip of rippled's book until an EXECUTING pass steps onto it — every later pass re-anchors tryAMM on ITS quality, even a pass that then goes dry. #106010546 7E746D30: the 11536 tip is the taker's own; iter 1 ...
- **fix(xrpl-ledger): crossing tail AMM turn is gated by strand admission (anchored-at-limit ub × trIn)** (8f34f5c , xrpl-validator) BookOfferCrossingStep::qualityUpperBound for an AMM-bearing strand with no better CLOB tip is the quality of the slice anchored AT the limit (qualityThreshold_, the inflated limitQuality), GROSSED by trIn — the book step charges the IN issuer's rate on the taker's spend even though the pool sees net amounts. When that bound misses the limit, activateNext drops the strand and the pool never fills the tail.
- **fix(xrpl-ledger): the offer stream keeps reaping past the limit once the strand was built** (60f425d , xrpl-validator) rippled's offer stream has no quality gate for STEPPING: once the strand was BUILT (tip or pool within the limit at some level), rev keeps stepping past DEAD offers on levels beyond the limit — reaping them — until a LIVE offer stops it (checkQualityThreshold ends the walk at execution, not at step). Our walk broke out at the first beyond-threshold level, leaving mainnet's reaps missing.
- **fix(xrpl-ledger): walk funding prices the reserve at the maker's first-peek OwnerCount** (cf9f395 , xrpl-validator) During a flow, offer deletions are DEFERRED — rippled queues crossed offers as removable (OfferCreate.cpp:460) and stale ones in ofrsToRm (StrandFlow.h:694) — so within the walk an owner's OwnerCount, hence reserve, keeps its pre-walk value for every funding check. Our sandbox deletes inline, freeing one owner-reserve unit (200000) per deletion.
- **fix(xrpl-ledger): bridge peeks reap dead offers they step over once the strand is running** (0a25745 , xrpl-validator) Once a bridged crossing has EXECUTED a fill, the strand is definitely built and every subsequent

stream-step removes the dead offers it passes — rippled reaps them even in an iteration whose liquidity then comes from the AMM. #106093637 1BE79D4A: iter 0 crosses the legB head (rD8VchnJ, funded exactly 0.59584264555 RLUSD), iter 1's stream steps over the EXPIRED rsqztuzpo (exp 839271795 ≤ parent close 839271841) and ...

- **fix(xrpl-ledger): a tfSell keeps taking pool liquidity after the pays minimum is met** (09be43a , xrpl-validator) The AMM consume gates bailed on `me_is_zero(rem_pays)` — but for a tfSell the pays side is only a MINIMUM: once met it saturates to zero while the sell keeps spending `rem_gets`, and rippled's flow keeps taking pool liquidity for the remaining IN (its `remainingOut` is not the pays side for a sell).
- **fix(xrpl-ledger): tail admission is the residual-tip interval, not the anchored-slice quality** (46a9986 , xrpl-validator) The anchored-at-limit slice test (8f34f5c) was the wrong quantity for the `maxOffer` branch: #106211626 D322E925 (BTC.Bitstamp → XRP) has no crossable tip, rippled's `ub` is `spot × trIn = 1.58397e-11 ≤ limit 1.58531e-11` — admitted, limitOut-sized pool fill, realised exactly at the limit — and the slice test's `trIn-grossed 1.5855e-11` vetoed it into a rest (4v7, plus the 1106203258 OfferCancel cascade).
- **fix(xrpl-node): the probe recognizes threading-only ModifiedNodes that carry FinalFields** (520fc6b , xrpl-validator) rippled fills `PreviousFields` only when a field actually changed, so a `ModifiedNode` with `FULL FinalFields` and null `PreviousFields` is a pure threading refresh (`PreviousTxnID/LgrSeq`). `trustDelete` threads BOTH owners' roots: #106065267 F9E4D516 redeems EXP to its issuer, the zeroed line is deleted, and the ISSUER's `AccountRoot` appears `Modified` with every field identical to the pre-tx state. The fixture format keeps ...

Docs and operations

- **docs: DirectStepI design — rippling through accounts, composed with book steps** (b35031a , xrpl-validator)

XRPL research

Automated research — candidate amendment ideas

Ideas generated and prior-art-checked by the XRPL Amendment Board, an automated research tool (read-only board: iceland.halcyon-names.io). They are exploration leads, not proposals.

- **Non-Custodial NFT Usage Leases** (gaming) Add a new owner-reserved `ltNFTOKEN_LEASE` ledger object (`keylet::nftLease(NFTokenID)`) created by a new `NFTokenLease` transaction carrying `sfNFTokenID`, s
- **Dynamic MPT Fund NAV & Redemption** (rwa) Extend `MPTokenIssuance`, already dynamically-updatable under `DynamicMPT`, with `sfNAVPerShare` and `sfRecordDate` `SFields` settable only by the issuer via th
- **Pseudo-Account Multilateral Netting Pools** (settlement) Add an owner-reserved `NettingPool` pseudo-account (XLS-64-style, deterministic `keylet::nettingPool(creator, seq)`, no master key, unowned by any single

Appendix — commits

Time	Repo	Commit	Subject
00:14	xrpl-validator	442c34c	fix(xrpl-ledger): TrustSet destination rules — no new lines at pseudo-accounts or opted-out issuers

Time	Repo	Commit	Subject
00:27	xrpl-validator	8c5e86b	fix(xrpl-ledger): AMMWithdraw judges the DERIVED burn against the position, and the one-asset minimum
00:48	xrpl-validator	24d983a	fix(xrpl-ledger): AMMCreate preclaim — duplicate pool, reserve, funds, LP-token sides
01:03	xrpl-validator	976d6e3	fix(xrpl-ledger): AccountDelete has no OwnerCount rule — the directory walk decides
01:24	xrpl-validator	b35031a	docs: DirectStepI design — rippling through accounts, composed with book steps
01:59	xrpl-validator	f13ad50	feat(xrpl-ledger): DirectStepI stage 1 — pure-account strands ripple for real
08:22	xrpl-validator	87bffeb	feat(xrpl-ledger): DirectStepI stage 2 — direct runs composed with book hops
08:57	xrpl-validator	4771bdd	feat(xrpl-ledger): the default path of a cross-issuer same-currency payment is a direct strand
11:00	xrpl-validator	0d4ce0f	fix(xrpl-ledger): offer crossing judges the pass with the OUT issuer's transfer rate composed
13:49	xrpl-validator	8d8079d	fix(xrpl-ledger): single-path crossing lets the AMM emit unbounded when the limit beats the LOB tip
15:23	xrpl-validator	6f32591	fix(xrpl-ledger): a skipped self-offer stays the tail AMM anchor; the trOut judge compose was wrong
15:38	xrpl-validator	8f34f5c	fix(xrpl-ledger): crossing tail AMM turn is gated by strand admission (anchored-at-limit $ub \times trIn$)
16:19	xrpl-validator	60f425d	fix(xrpl-ledger): the offer stream keeps reaping past the limit once the strand was built
16:51	xrpl-validator	c9f395	fix(xrpl-ledger): walk funding prices the reserve at the maker's first-peek OwnerCount
17:37	xrpl-validator	0a25745	fix(xrpl-ledger): bridge peeks reap dead offers they step over once the strand is running
17:57	xrpl-validator	09be43a	fix(xrpl-ledger): a tfSell keeps taking pool liquidity after the pays minimum is met
18:46	xrpl-validator	46a9986	fix(xrpl-ledger): tail admission is the residual-tip interval, not the anchored-slice quality
20:12	xrpl-validator	520fc6b	fix(xrpl-node): the probe recognizes threading-only ModifiedNodes that carry FinalFields